DESENVOLVIMENTO MOBILE · MÓDULO 10 · TRILHA FLUTTER · NÍVEL INTERMEDIÁRIO

Persistência Local

shared_preferences, armazenamento seguro, arquivos, sqflite e Drift com migrações e testes

Prof. Dr. Francisco dos Santos Viana

São Luís – Maranhão – Brasil · 2026

Objetivos do módulo

Ao final deste módulo você será capaz de

- Escolher onde guardar cada tipo de dado no dispositivo.
- Salvar preferências com `SharedPreferencesAsync` e segredos com `flutter_secure_storage`.
- Ler e gravar arquivos com `path_provider`.
- Usar SQLite diretamente com **sqflite**.
- Modelar tabelas, consultas reativas e migrações com **Drift**.
- Tornar o banco local a fonte da verdade de um app *offline-first*.

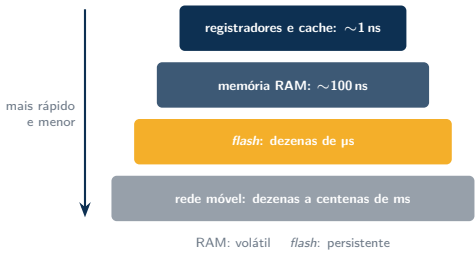
Roteiro

Neste módulo

- Fundamentos: persistência, cache e sincronização
- Onde guardar cada dado
- Chave-valor, segredos e arquivos
- SQLite com sqflite
- Drift: SQLite tipado e reativo
- Banco local na arquitetura
- Fechamento

Fundamentos: persistência, cache e sincronização

Memória e armazenamento: velocidade *versus* permanência



Persistência

Guardar dados de forma que sobrevivam ao fim do processo, ao desligar do aparelho e à falta de rede. No celular, isso significa gravar na memória *flash*.

Por que persistir em um app

O sistema encerra apps em segundo plano (Módulo 07), a rede falha e o usuário espera abrir o app e ver seus dados na hora.

A memória *flash*

Grava em blocos e se desgasta com o uso. Muitas gravações pequenas custam mais que uma grande: agrupe operações em transações (Módulo 09).

Onde o sistema guarda os arquivos do app

Finalidade	Android	iOS	Flutter (`path_provider`)
Dados internos do app	`filesDir`, `databases/`	`Library/Application Support`	`getApplicationSupportDirectory()`
Documentos do usuário	`filesDir` ou MediaStore	`Documents`	`getApplicationDocumentsDirectory()`
Cache descartável	`cacheDir`	`Library/Caches`	`getApplicationCacheDirectory()`
Temporários	`cacheDir`	`tmp`	`getTemporaryDirectory()`

Sandbox de arquivos

Cada app tem pastas privadas que outros apps não leem. Desinstalar o app apaga tudo.

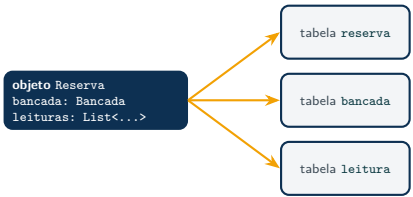
Cópia de segurança

O Android copia até 25 MB por app para a conta Google; o iOS inclui `Documents` e `Library`, exceto `Caches`.

Cache some

O sistema apaga caches quando falta espaço. Nunca guarde ali o que não puder baixar de novo.

Do objeto à tabela



referências viram chaves estrangeiras; listas viram outra tabela

Descompasso objeto-relacional

Objetos têm referências, listas e herança; tabelas têm linhas, colunas e chaves. Traduzir de um mundo para o outro é o trabalho do mapeamento.

Abordagem	Como é
SQL à mão	você escreve o SQL e converte as linhas (`sqflite`)
SQL verificado	você escreve o SQL; a ferramenta confere na compilação e gera o código (Room, Drift)
ORM completo	a biblioteca gera o SQL a partir dos objetos
Banco de objetos	guarda objetos sem tabelas (ObjectBox, Realm)

Dica

Room e Drift ficam no meio-termo: SQL explícito, mas com tipos e erros apontados antes de o app rodar.

O problema N+1

Buscar uma lista e depois uma consulta por item gera N+1 idas ao banco. Use `JOIN` ou relações carregadas de uma vez.

O esquema também tem versões



Migração

Programa que leva o banco de uma versão do esquema para a seguinte **sem perder os dados** do usuário. Cada versão publicada do app pode encontrar qualquer versão anterior do banco.

Regras de ouro

Migrações só andam para a frente e nunca são editadas depois de publicadas; teste a partir de **cada** versão antiga; apagar e recriar o banco só serve para dados de cache.

Cache: perto e rápido, mas pode estar velho

Cache

Cópia de dados guardada perto de quem usa para evitar buscá-los de novo. **Acerto**: o dado estava lá. **Falta**: é preciso ir à fonte.

Estratégia	Como funciona
Cache primeiro	usa o cache; só vai à rede se não houver
Rede primeiro	tenta a rede; usa o cache se falhar
Mostra e revalida	mostra o cache já e atualiza em seguida (*stale-while-revalidate*)
Só rede	dados que não podem estar velhos (saldo, reserva)

Invalidação

Decidir quando o cache deixou de valer: por tempo (TTL), por evento (o servidor avisou) ou por versão (*ETag*, Módulo 11).

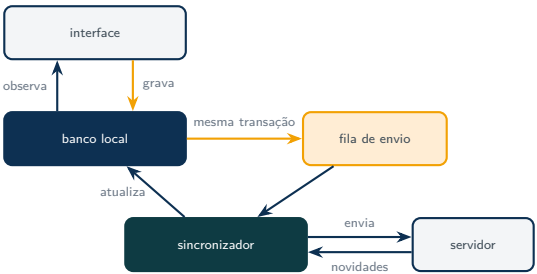
Quem sai quando enche

A política mais comum é a LRU (*least recently used*): descarta o item usado há mais tempo. Imagens em memória costumam usar esse critério.

Um pouco de história

Phil Karlton brincava que só há duas coisas difíceis na computação: invalidar cache e dar nomes às coisas.

Offline-first: o banco local como fonte da verdade



Offline-first

A interface lê e grava **sempre** no banco local. Um sincronizador troca dados com o servidor quando há rede. Sem rede, o app continua funcionando.

Fila de envio (*outbox*)

Cada alteração local vira um registro pendente. O sincronizador envia em ordem, com novas tentativas e espera crescente, mesmo com o app fechado (WorkManager, `workmanager`).

Atenção

Reenvios podem duplicar operações: mande uma chave de idempotência para o servidor reconhecer repetições (Módulo 17).

Sincronização e conflitos

O conflito

Ana e Bruno, sem internet, reservam a bancada B3 para as 14 h. Quando a rede volta, as duas reservas chegam ao servidor. Quem fica com a bancada?

Estratégia	Quando usar
Última gravação vence	dados pessoais pouco disputados; depende de relógios confiáveis
Regra no servidor	recursos disputados: a segunda reserva é recusada
Mesclagem por campo	edições em campos diferentes do mesmo registro
Perguntar ao usuário	textos editados dos dois lados
CRDT	dados colaborativos que se mesclam sozinhos

Concorrência otimista

Cada registro tem uma versão. Quem grava informa a versão que leu; se outra pessoa gravou antes, o servidor recusa e o app resolve o conflito.

Na interface

Mostre o que está pendente de envio e avise com clareza quando uma operação for recusada. Ações que não podem esperar (pagar, reservar) devem exigir rede.

Dados em repouso: criptografia e segredos

Criptografia em repouso

Os dados gravados ficam cifrados e só são decifrados com uma chave. Quem copia o armazenamento sem a chave vê apenas bytes sem sentido.

Camada	O que protege
Sistema	todo o armazenamento: criptografia por arquivo (Android) e *Data Protection* (iOS)
Cofre de chaves	chaves e segredos em hardware: Keystore (Android), Keychain e Secure Enclave (iOS)
App	banco cifrado (SQLCipher) e campos sensíveis cifrados

Por que o hardware importa

Chaves criadas no Keystore ou no Secure Enclave não saem do chip: o app pede "cifre isto" e recebe o resultado, sem nunca ver a chave.

O que nunca fazer

Guardar senhas; guardar *tokens* em preferências comuns; deixar chaves de API fixas no código, que pode ser extraído do pacote publicado.

Dica

Pense no risco: aparelho perdido, aparelho com *root* e cópias de segurança na nuvem.

Onde guardar cada dado

Opções de armazenamento

Opção	Para quê	Exemplo no LabMonitor
`shared_preferences`	pares chave-valor simples	tema escuro, último laboratório aberto
`flutter_secure_storage`	segredos cifrados (Keychain/Keystore)	token de acesso, chave da API
Arquivos (`path_provider`)	documentos, imagens, exportações	relatório CSV, fotos de equipamentos
`sqflite`	SQLite com SQL escrito à mão	tabelas simples, controle total do SQL
Drift	SQLite tipado, reativo, com migrações	histórico de leituras dos sensores

Atenção

Nunca guarde tokens ou senhas em `shared_preferences`: o arquivo não é cifrado.

Dica

Bancos NoSQL locais (ObjectBox, Hive CE) também existem; confira a manutenção do pacote no pub.dev antes de adotar.

Chave-valor, segredos e arquivos

Preferências com `shared_preferences`

data/services/preferencias_service.dart

```dart
import 'package:shared_preferences/shared_preferences.dart';

class PreferenciasService {
  final _prefs = SharedPreferencesAsync();

  Future<bool> modoEscuro() async =>
      await _prefs.getBool('modoEscuro') ?? false;

  Future<void> definirModoEscuro(bool v) =>
      _prefs.setBool('modoEscuro', v);

  Future<String?> ultimoLab() =>
      _prefs.getString('ultimoLab');

  Future<void> limpar() => _prefs.clear();
}
```

Duas APIs atuais

- `SharedPreferencesAsync`: toda leitura vai à plataforma; sempre atualizada.
- `SharedPreferencesWithCache`: carrega um conjunto de chaves (`allowList`) e lê de forma síncrona.

Dica

A API antiga `SharedPreferences.getInstance()` ainda funciona, mas a documentação recomenda as duas novas.

Segredos e arquivos

Armazenamento seguro

```
const _seguro = FlutterSecureStorage();

Future<void> salvarToken(String jwt) =>
    _seguro.write(key: 'token', value: jwt);

Future<String?> lerToken() =>
    _seguro.read(key: 'token');

Future<void> sair() =>
    _seguro.delete(key: 'token');
```

Por baixo

iOS: **Keychain**. Android: chave no **Android Keystore** cifrando os dados. Web: sem garantia equivalente.

Arquivos com path_provider

```
Future<File> _arquivo() async {
  final dir = await
      getApplicationDocumentsDirectory();
  return File('${dir.path}/leituras.csv');
}

Future<void> anexar(Leitura l) async {
  final f = await _arquivo();
  await f.writeAsString(
    '${l.em.toIso8601String()};${l.valor}\n',
    mode: FileMode.append);
}
```

Dica

`getTemporaryDirectory()` para cache que o sistema pode apagar; `getApplicationSupportDirectory()` para arquivos internos.

SQLite com sqflite

Abrindo o banco e criando tabelas

data/services/banco_service.dart

```dart
import 'package:path/path.dart';
import 'package:sqflite/sqflite.dart';

Future<Database> abrirBanco() async => openDatabase(
  join(await getDatabasesPath(), 'labmonitor.db'),
  version: 2,
  onCreate: (db, versao) async {
    await db.execute('''
      CREATE TABLE leitura (
        id     INTEGER PRIMARY KEY AUTOINCREMENT,
        sensor TEXT NOT NULL,
        valor  REAL NOT NULL,
        em     INTEGER NOT NULL)''');
    await db.execute(
      'CREATE INDEX idx_leitura ON leitura(sensor, em)');
  },
  onUpgrade: (db, de, para) async {
    if (de < 2) await db.execute(
      'ALTER TABLE leitura ADD COLUMN unidade TEXT');
  },
);
```

Ciclo de versões

`onCreate` roda na primeira instalação; `onUpgrade` roda quando a `version` aumenta. Cada mudança de esquema ganha um bloco `if (de < N)`.

Atenção

Em instalações novas o `onCreate` precisa criar o esquema **mais recente** (inclua aqui a coluna `unidade`).

Dica

Datas como `INTEGER` (milissegundos desde 1970) ordenam e comparam rápido.

Consultas, transações e segurança

CRUD com sqflite

```dart
Future<int> inserir(Database db, Leitura l) =>
    db.insert('leitura', l.paraMapa());

Future<List<Leitura>> ultimas(Database db, String s) async {
  final linhas = await db.query('leitura',
      where: 'sensor = ?', whereArgs: [s],
      orderBy: 'em DESC', limit: 50);
  return linhas.map(Leitura.deMapa).toList();
}

Future<void> importar(Database db, List<Leitura> ls) =>
    db.transaction((txn) async {
      final lote = txn.batch();
      for (final l in ls) lote.insert('leitura', l.paraMapa());
      await lote.commit(noResult: true);
    });
```

SQL injection

`db.rawQuery("... WHERE sensor = '$s'")` deixa o usuário escrever SQL. Use sempre `?` com `whereArgs` ou `arguments`.

Transação + batch

Tudo ou nada (ACID, Módulo 09) e muito mais rápido do que mil `insert` isolados.

Drift: SQLite tipado e reativo

Definindo tabelas e o banco

data/services/banco.dart

```dart
import 'package:drift/drift.dart';
import 'package:drift_flutter/drift_flutter.dart';
part 'banco.g.dart';

@TableIndex(name: 'idx_leitura', columns: {#sensor, #em})
class Leituras extends Table {
  IntColumn get id => integer().autoIncrement()();
  TextColumn get sensor => text().withLength(max: 40)();
  RealColumn get valor => real()();
  DateTimeColumn get em => dateTime()();
}

@DriftDatabase(tables: [Leituras])
class BancoApp extends _$BancoApp {
  BancoApp([QueryExecutor? e])
      : super(e ?? driftDatabase(name: 'labmonitor'));

  @override
  int get schemaVersion => 1;
}
```

```
$ flutter pub add drift drift_flutter
$ flutter pub add -d drift_dev build_runner
$ dart run build_runner build
```

O que é gerado

A classe de dados `Leitura`, o `LeiturasCompanion` para inserções e as consultas tipadas. Erros de nome de coluna viram erros de compilação.

Dica

O construtor opcional permite passar um banco em memória nos testes.

Consultas e streams

Dentro de BancoApp (ou de um DAO)

```
Future<int> registrar(String sensor, double v) =>
    into(leituras).insert(LeiturasCompanion.insert(
        sensor: sensor, valor: v, em: DateTime.now()));

Stream<List<Leitura>> observar(String sensor) =>
    (select(leituras)
          ..where((t) => t.sensor.equals(sensor))
          ..orderBy([(t) => OrderingTerm.desc(t.em)])
          ..limit(50))
        .watch();

Future<int> apagarAntigas(DateTime limite) =>
    (delete(leituras)
          ..where((t) => t.em.isSmallerThanValue(limite)))
        .go();

Future<double?> media(String sensor) => customSelect(
    'SELECT AVG(valor) AS m FROM leituras WHERE sensor = ?',
    variables: [Variable(sensor)], readsFrom: {leituras},
  ).map((r) => r.read<double?>('m')).getSingle();
```

`.watch()`

Devolve um `Stream` que emite de novo sempre que a tabela muda. A tela se atualiza sozinha quando chega uma leitura nova.

Dica

Também é possível escrever as consultas em SQL puro em arquivos `.drift`; o Drift as valida e gera métodos Dart.

Migrações com Drift

Versão 2: nova coluna

```dart
class Leituras extends Table {
  // ... colunas anteriores
  TextColumn get unidade => text().nullable()();
}

@override
int get schemaVersion => 2;

@override
MigrationStrategy get migration => MigrationStrategy(
  onCreate: (m) => m.createAll(),
  onUpgrade: (m, de, para) async {
    if (de < 2) {
      await m.addColumn(leituras, leituras.unidade);
    }
  },
);
```

```
$ dart run drift_dev make-migrations
```

Migrações verificadas

O comando salva o esquema de cada versão e gera migrações passo a passo e testes que conferem se a versão 1 migrada é igual à versão 2 criada do zero.

Atenção

Uma migração errada pode apagar dados de quem já usa o app. Teste sempre a atualização, não só a instalação nova.

Banco local na arquitetura

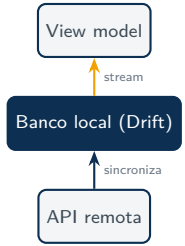
Repositório *offline-first*

data/repositories/leitura_repository.dart

```dart
class LeituraRepository {
  LeituraRepository({required this.banco, required this.api});
  final BancoApp banco;
  final ApiService api;

  // a UI sempre lê do banco local
  Stream<List<Leitura>> observar(String sensor) =>
      banco.observar(sensor);

  // a rede só atualiza o banco
  Future<Result<void>> sincronizar(String sensor) async {
    try {
      final novas = await api.leituras(sensor);
      await banco.batch((b) => b.insertAll(banco.leituras,
          novas.map((d) => d.paraCompanion())));
      return const Ok(null);
    } on Exception catch (e) { return Falha(e); }
  }
}
```



Fonte única da verdade

A tela observa o banco; a rede apenas o atualiza. Sem internet, o app continua mostrando os últimos dados.

Testando o banco

test/banco_test.dart

```dart
import 'package:drift/native.dart';
import 'package:flutter_test/flutter_test.dart';

void main() {
  late BancoApp banco;
  setUp(() => banco = BancoApp(NativeDatabase.memory()));
  tearDown(() => banco.close());

  test('observar emite a leitura registrada', () async {
    await banco.registrar('temp', 23.5);
    final lista = await banco.observar('temp').first;
    expect(lista.single.valor, 23.5);
  });
}
```

Banco em memória

`NativeDatabase.memory()` cria um SQLite real, descartado a cada teste. Rápido e sem efeitos colaterais.

Web

`sqflite` não roda na web. O Drift funciona na web com WebAssembly, mas exige configuração extra.

sqflite ou Drift?

	sqflite	Drift
Consultas	SQL em strings	API Dart tipada ou SQL validado
Erros de esquema	em tempo de execução	em tempo de compilação
Reatividade	manual	`.watch()` embutido
Migrações	`onUpgrade` manual	`MigrationStrategy` + verificação gerada
Geração de código	não	sim (`build_runner`)
Equivalente Kotlin	SQLite direto	Room

Para aprender SQL, comece com **sqflite**. Para apps reais com várias tabelas e telas reativas, prefira **Drift**.

Fechamento

Resumo

Resumo

- Preferências em `shared_preferences`; segredos em `flutter_secure_storage`; documentos em arquivos.
- `sqflite` dá acesso direto ao SQLite: use `?`, transações e `batch`.
- Drift gera código tipado, oferece consultas reativas e migrações verificadas.
- O banco local é a fonte da verdade; a rede apenas o sincroniza.
- Teste com banco em memória e teste também as migrações.

Glossário (1/2)

Termo	Significado
Persistência	guardar dados que sobrevivem ao fim do processo e ao desligar
***Sandbox* de arquivos**	pastas privadas do app, apagadas na desinstalação
`shared_preferences`	pares chave-valor simples, sem criptografia
`flutter_secure_storage`	segredos cifrados no Keychain ou com o Keystore
`path_provider`	localiza as pastas do app em cada plataforma
`sqflite`	acesso ao SQLite com SQL escrito à mão
Drift	SQLite tipado e reativo, com código gerado
Migração	leva o banco de uma versão do esquema à seguinte sem perder dados
Problema N+1	uma consulta para a lista e mais uma para cada item

Glossário (2/2)

Termo	Significado
Cache	cópia guardada perto de quem usa, para não buscar de novo
Invalidação	decidir quando o cache deixou de valer (TTL, evento, versão)
LRU	política que descarta o item usado há mais tempo
Offline-first	a interface lê e grava no banco local; a sincronização vem depois
Fila de envio	alterações locais pendentes de envio ao servidor (*outbox*)
Concorrência otimista	a gravação é recusada se a versão lida já mudou
Keystore e Keychain	cofres de chaves do Android e do iOS, protegidos por hardware

Questões de revisão

1. Por que um app precisa persistir dados mesmo quando tem internet?
2. Onde guardar a preferência de tema, o *token* de acesso, o histórico de leituras e as miniaturas baixadas?
3. Quando preferir Drift a `sqflite`?
4. Um usuário atualiza da versão 1 direto para a 4 do app. O que a migração do Drift precisa tratar?
5. Explique o problema N+1 e como evitá-lo.
6. Descreva o caminho de uma reserva feita sem internet em uma arquitetura *offline-first*.
7. Dois alunos reservam a mesma bancada sem internet. Que estratégia de conflito usar e por quê?
8. Por que não guardar o *token* em `shared_preferences`?

Exercícios

Exercício 1 — Preferências

Salve o tema escolhido e o último laboratório aberto e restaure-os ao abrir o app.

Exercício 2 — sqflite

Crie as tabelas `laboratorio` e `equipamento` (1:N) e liste os equipamentos com um `JOIN`.

Exercício 3 — Migração

Adicione a coluna `unidade` na versão 2 do banco Drift e escreva um teste que migra da versão 1.

LabMonitor — Etapa 10

Histórico de leituras em Drift observado pelo painel, preferências com `SharedPreferencesAsync` e token em armazenamento seguro.

Para aprofundar

- Persistência no Flutter: `https://docs.flutter.dev/cookbook/persistence`
- shared_preferences: `https://pub.dev/packages/shared_preferences`
- flutter_secure_storage: `https://pub.dev/packages/flutter_secure_storage`
- sqflite: `https://pub.dev/packages/sqflite`
- Drift: `https://drift.simonbinder.eu`

Respostas das questões de revisão

1. O sistema encerra apps em segundo plano, a rede falha e o usuário quer ver os dados na hora: o banco local serve de cache e de fonte da verdade.
2. Tema: `shared_preferences`. *Token*: `flutter_secure_storage`. Leituras: Drift ou `sqflite`. Miniaturas: `getApplicationCacheDirectory()`.
3. Quando se quer consultas tipadas e verificadas na geração de código, *streams* reativas e migrações assistidas; `sqflite` atende bancos pequenos com controle total do SQL.
4. O `onUpgrade` deve aplicar todos os passos de `de` até `para` em sequência (2, 3 e 4), e cada passo precisa de teste a partir da versão antiga (`make-migrations`).
5. Buscar a lista e depois fazer uma consulta por item gera N+1 idas ao banco. Use um `JOIN` ou carregue os itens relacionados de uma vez.
6. A reserva é gravada no banco como pendente, junto com um registro na fila de envio, na mesma transação. A tela já a mostra; uma tarefa em segundo plano a envia quando houver rede e atualiza o estado.
7. Regra no servidor: a bancada é um recurso disputado. A primeira reserva confirmada vence, a segunda é recusada e o app avisa o usuário.
8. O arquivo não é cifrado pelo app e pode ir para cópias de segurança ou ser lido em aparelhos com *root*. Segredos vão para o Keychain ou para o Keystore, via `flutter_secure_storage`.

Fim do Módulo 10

Próximo: Módulo 11 — Rede e APIs: HTTP, JSON, `http` e Dio.